

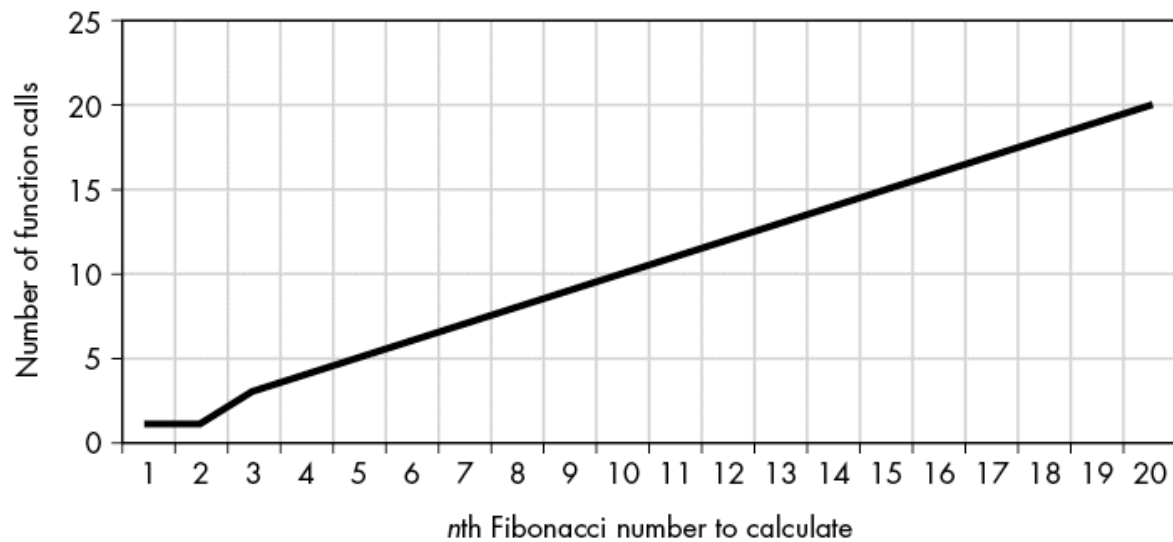


Figure 7-2: The number of function calls sharply increases for the original `fibonacci()` function (top) but grows only slowly for the memoized `fibonacci()` function (bottom).

The Python version of the memoized Fibonacci algorithm is in *fibonacciByRecursionMemoized.py*. The additions to the original *fibonacciByRecursion.html* program from Chapter 2 have been marked in bold:

```

fibonacciCache = {} ❶ # Create the global cache.

def fibonacci(nthNumber, indent=0):
    global fibonacciCache
    indentation = '.' * indent
    print(indentation + 'fibonacci(%s) called.' % (nthNumber))

    if nthNumber in fibonacciCache:
        # If the value was already cached, return it.
        print(indentation + 'Returning memoized result: %s' % (fibonacciCache[nthN
umber]))
        return fibonacciCache[nthNumber] ❷

    if nthNumber == 1 or nthNumber == 2:
        # BASE CASE
        print(indentation + 'Base case fibonacci(%s) returning 1.' % (nthNumber))
        fibonacciCache[nthNumber] = 1 ❸ # Update the cache.
        return 1
    else:
        # RECURSIVE CASE
        print(indentation + 'Calling fibonacci(%s) (nthNumber - 1).' % (nthNumber

```